

开发者测试理论

东南大学计算机科学与工程学院

李必信

主要内容

- 1 开发者测试
- 2 测试过程(面向测试任务)
- 3 测试方法
- 4 测试类型
- 5 测试工具

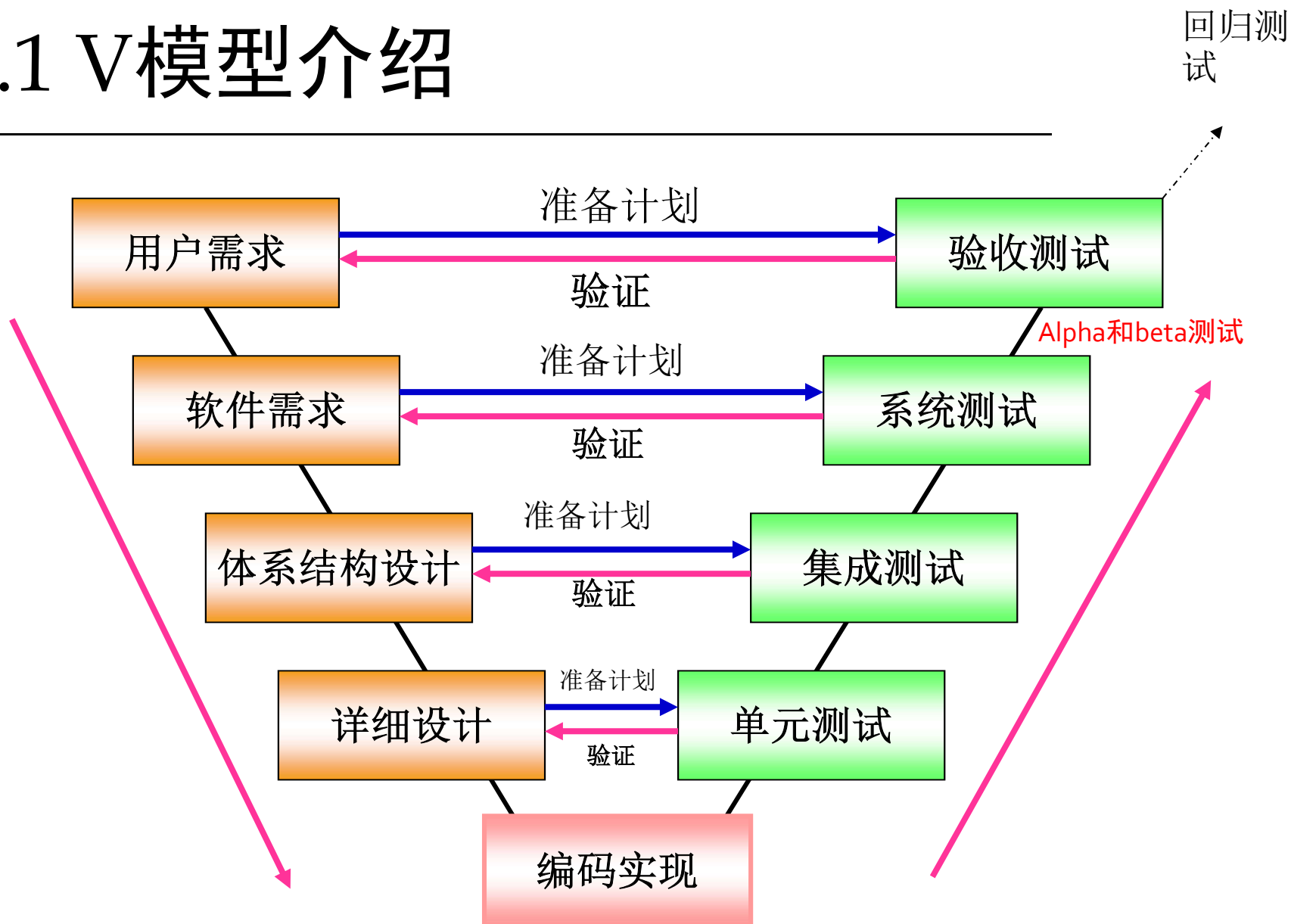
1. 开发者测试

- 开发者测试（Developer Testing, DT），是指开发者所做的测试，有别于专职测试人员(来自测评机构)进行的测试活动。DT目标是在软件交付（验收测试）前，发现和解决绝大多数代码缺陷，而其理论依据是业界研究反复揭示的“前端发现问题的代价远小于后端”。

以V模型为例

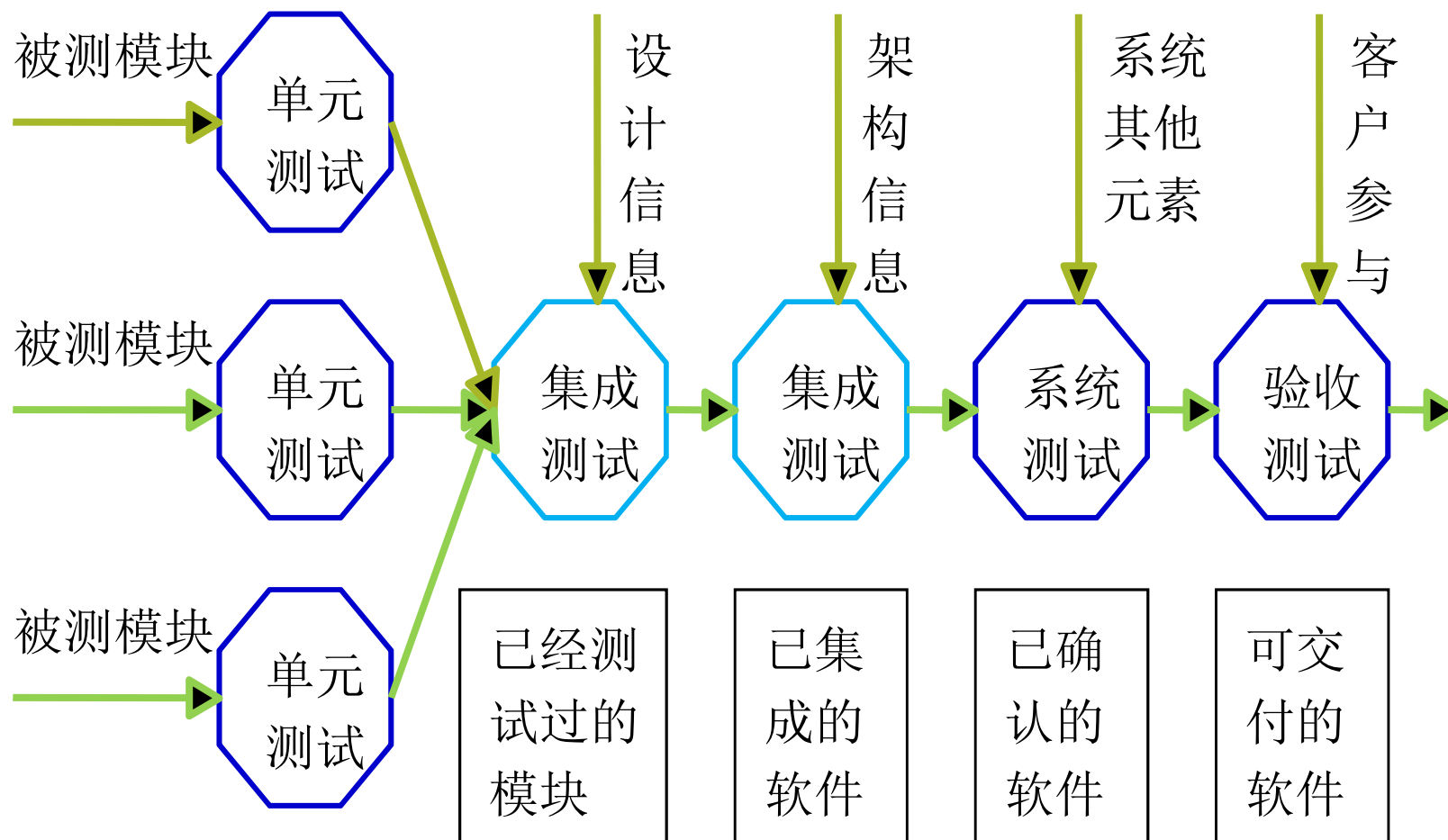
- 单元测试
- 集成测试
- 系统测试
- 验收测试
- Alpha和beta测试
- 回归测试

1.1 V模型介绍



测试阶段的信息流

除了测试本身的信息外，开发阶段的一些内容也是测试的输入。



1.2 单元测试

- **单元测试**是软件开发中的一种测试技术，旨在**验证**代码中最小可独立测试单元（如函数、方法、类）**的行为是否符合预期**。
- **核心概念**
 - ✓ **测试对象**：针对代码中的“单元”，通常指一个函数、方法或类。
 - ✓ **独立性**：测试时隔离外部依赖（如数据库、网络服务），使用模拟（Mock）或桩（Stub）技术，确保测试**仅关注当前单元的逻辑**。
 - ✓ **自动化**：通过测试框架（如JUnit、pytest、Jest）自动执行，便于集成到开发流程中。

实施步骤

1. 编写测试用例：

- ✓ 覆盖正常输入、异常输入（如非法参数）、边界条件（如极值、空值）。
- ✓ 示例：测试加法函数 `add(a, b)`，需验证 $2+3=5$ 、负数相加 $(-1)+(-1)=-2$ 、非数字输入抛出错误等。

2. 运行测试：

- ✓ 使用框架命令（如 `pytest`、`npm test`）执行测试。

3. 验证结果：

- ✓ 断言（Assert）预期结果与实际结果是否一致。
- ✓ 测试失败时，框架会报告具体错误位置。

工具与框架

- Python: `unittest`、`pytest`
- Java: `JUnit`、`TestNG`
- JavaScript: `Jest`、`Mocha`
- C++: `Google Test`

与其他测试的区别

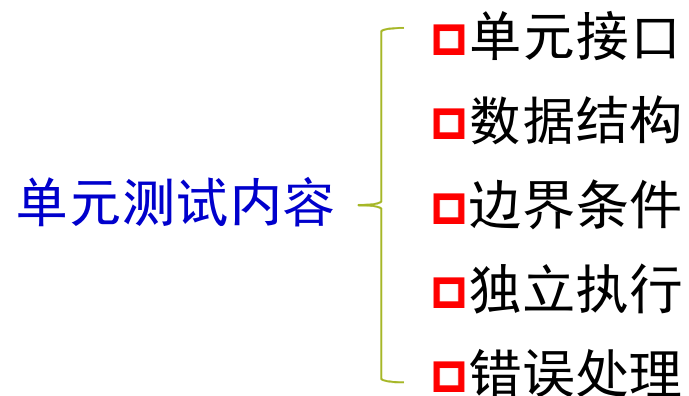
- 集成测试：验证多个模块协作，涉及真实依赖（如数据库）。
- 系统测试：评估整个系统是否符合需求。

FIRST原则：

- ✓ Fast（快速）：测试应在毫秒级完成。
- ✓ Isolated（独立）：用例间无依赖，可单独运行。
- ✓ Repeatable（可重复）：在任何环境结果一致。
- ✓ Self-Validating（自验证）：自动判断成功/失败。
- ✓ Timely（及时）：测试与代码同步编写。

高覆盖率：可以利用工具（如Coverage.py、Istanbul）统计代码覆盖率，但避免盲目追求100%，注重关键逻辑覆盖。

可以是黑盒测试，也可以是白盒测试



例子(1/5)

□ 模块接口测试

- 检查进出模块的数据是否正确
- Checklist:
 - 模块的实际输入与定义的输入是否一致
 - 个数、类型、顺序
 - 使用其他模块时，是否检查可用性和处理结果
 - 使用外部资源时，是否检查可用性并及时释放资源
 - 内存、文件、硬盘、端口等
 - 其他

示例：计算最大公约数（GCD）的模块

假设有一个函数 `calculate_gcd(a, b)`，用于计算两个整数的最大公约数。

1. 参数测试

• 测试用例 1：参数数量检查

- 测试目的：验证调用时参数不足是否会触发错误
- 输入：`calculate_gcd(5)`（仅传一个参数）
- 预期输出：抛出 `TypeError`（参数缺失）

• 测试用例 2：参数类型检查

- 测试目的：验证非整数输入是否被正确处理
- 输入：`calculate_gcd("5", 10)`（字符串参数）
- 预期输出：抛出 `TypeError`（类型错误）

例子(2/5)

□模块局部数据结构测试

- 检查局部数据结构能否保持完整性
- Checklist:
 - 变量从来没有被使用
 - 可能别的地方使用了错误的变量名
 - 变量没有初始化
 - 错误的类型转换
 - 数组越界
 - 非法指针
 - 变量或函数名称拼写错误
 - 使用了外部变量或函数
 - 其他

数组越界的常见场景

1. 索引为负数 (如 `arr[-1]`)
2. 索引超过数组长度 (如 `arr[10]`, 数组长度为 5)
3. 循环条件错误 (如 `for (i=0; i<=arr.length; i++)`)

示例代码与测试用例

假设有一个函数 `get_element(arr, index)`, 用于返回数组中指定索引位置的元素。

1. 测试索引越界的异常行为

• 测试用例 1: 索引为负数

```
python 📄 复制代码  
  
# 示例代码 (Python)  
def get_element(arr, index):  
    return arr[index]  
  
# 测试代码  
def test_negative_index():  
    with pytest.raises(IndexError):  
        get_element([1, 2, 3], -1)
```

```
1
2 public class Test1 {
3
4     public static void main(String[] args) {
5         // TODO Auto-generated method stub
6         The value of the local variable var is not used int var;
7     }
8
9 }
10
```

```

1
2 public class Test1 {
3
4     public static void main(String[] args) {
5         // TODO Auto-generated method stub
6         int var;
7         if(var > 1) var++;
8
9
10    }
11

```

Multiple markers at this line

- The local variable var may not have been initialized
- The local variable var may not have been initialized

例子(3/5)

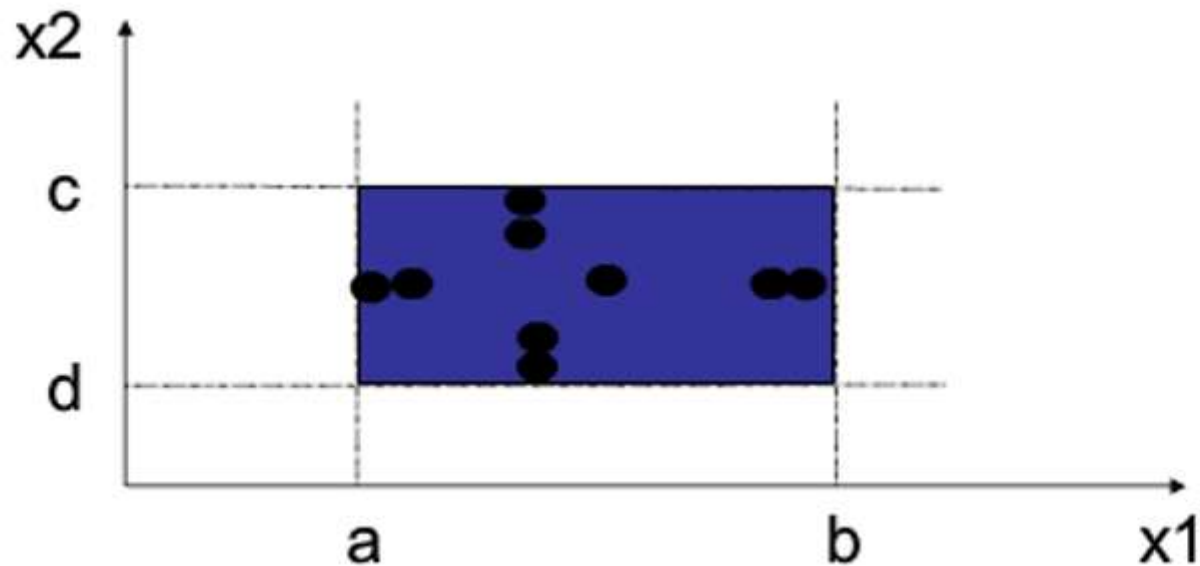
□ 模块边界条件测试

- 检查临界数据是否正确处理
- Checklist:
 - 普通合法数据是否正确处理
 - 普通非法数据是否正确处理
 - 边界内最接近边界的(合法)数据是否正确处理
 - 边界外最接近边界的(非法)数据是否正确处理
 - 其他

覆盖正常输入、异常输入（如非法参数）、边界条件（如极值、空值）

- 例1：有两个输入变量 $x_1(a \leq x_1 \leq b)$ 和 $x_2(c \leq x_2 \leq d)$ 的程序F的边界值分析测试用例如下：

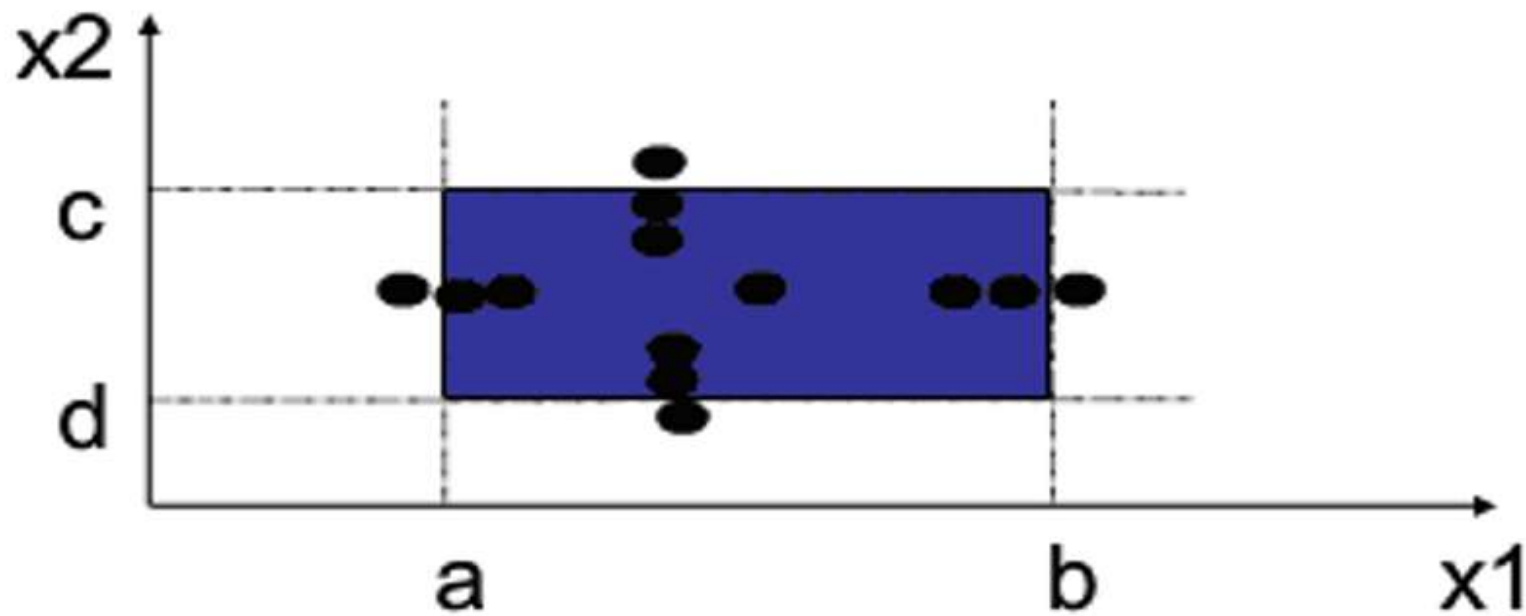
{ $\langle x_1\text{nom}, x_2\text{min} \rangle$, $\langle x_1\text{nom}, x_2\text{min}+ \rangle$, $\langle x_1\text{nom}, x_2\text{nom} \rangle$,
 $\langle x_1\text{nom}, x_2\text{max} \rangle$, $\langle x_1\text{nom}, x_2\text{max}- \rangle$, $\langle x_1\text{min}, x_2\text{nom} \rangle$,
 $\langle x_1\text{min}+, x_2\text{nom} \rangle$, $\langle x_1\text{max}, x_2\text{nom} \rangle$, $\langle x_1\text{max}-, x_2\text{nom} \rangle$ }



4n+1

覆盖正常输入、异常输入（如非法参数）、边界条件（如极值、空值）

前面例1中的测试用例按 $6n+1$ 公式扩充如下：



$\langle x1_{nom}, x2_{max+} \rangle, \langle x1_{nom}, x2_{min-} \rangle$

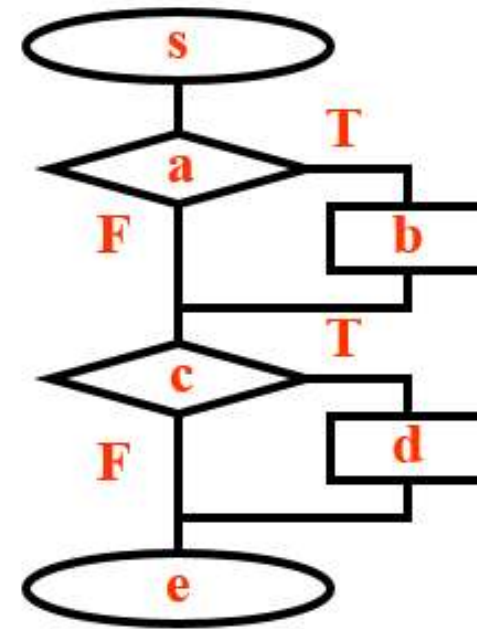
$\langle x1_{min-}, x2_{nom} \rangle, \langle x1_{max+}, x2_{nom} \rangle$

例子(4/5)

□模块独立执行路径测试

- 检查由于计算错误、判定错误、控制流错误导致的程序错误
- Checklist:
 - 死代码
 - 错误的计算优先级
 - 精度错误
 - 比较运算错误
 - 赋值错误
 - 表达式的不正确符号
 - $>$ 、 $>=$ 、 $=$ 、 $==$ 、 $!=$
 - 循环变量的使用错误
 - 错误赋值
 - 其他

```
procedure example(y,z:real;var x:real);  
begin  
  if (y>1) and (z=0) then x:=x/y;  
  if (y=2) or (x>1) then x:=x+1;  
end;
```



例子(5/5)

□ 模块内部错误处理测试

- 检查内部错误处理设施是否有效
- Checklist:
 - 是否检查错误出现
 - 资源使用前后
 - 其他模块使用前后
 - 出现错误，是否进行错误处理
 - 抛出错误、通知用户、进行记录
 - 错误处理是否有效
 - 在系统干预前处理
 - 报告和记录的错误真实详细
 - 其他

1.3 集成测试

集成测试（Integration Testing）是软件测试的关键阶段，旨在验证多个模块、组件或服务在协同工作时的交互是否符合预期。它关注的是单元之间的接口、数据传递、依赖关系和整体功能逻辑，确保组合后的系统部分能正确运行。

□为什么要进行集成测试？

- 一个模块可能对另一个模块产生不利的影响
- 将子功能合成时不一定产生所期望的主功能
- 独立可接受的误差，在组装后可能会超过可接受的误差限度
- 可能会发现单元测试中未发现的接口方面的错误
- 在单元测试中无法发现时序问题（实时系统）
- 在单元测试中无法发现资源竞争问题

核心概念

1. 测试对象：

- ✓ 模块间的接口（如函数调用、API 通信）。
- ✓ 子系统或微服务之间的交互（如数据库连接、网络请求）。
- ✓ 第三方服务集成（如支付网关、短信服务）。

2. 测试范围：

- ✓ 介于单元测试（验证单个模块）和系统测试（验证完整系统）之间。
- ✓ 可以是小规模（两个模块）或中规模（多个组件）。

3. 依赖管理：

- ✓ 部分依赖使用真实服务（如数据库），部分使用模拟工具（Mock）。 例如：测试订单服务时，使用真实数据库，但模拟支付接口。

核心目的

1. 暴露接口问题：发现模块间数据传输错误（如参数格式不符、返回值丢失）；检测资源竞争、死锁或并发问题。
2. 验证协作逻辑：确保模块组合后的功能符合设计（如用户注册后自动发送邮件）；发现未处理的异常或边界条件（如网络超时、数据一致性）。
3. 支持持续集成（CI）：在代码合并时自动运行，快速反馈集成问题。

集成测试工具与框架

- API 测试：Postman、RestAssured、Apifox
- 数据库测试：Testcontainers（用Docker创建临时数据库）
- 微服务测试：Pact（契约测试）、WireMock（模拟HTTP服务）
- 自动化框架：
 - ✓ Java: JUnit 5 + Spring Boot Test
 - ✓ Python: pytest + requests-mock
 - ✓ JavaScript: Jest + Supertest

示例：电商系统集成测试

假设该系统包含 用户服务 和 订单服务：

1. 场景：用户下单后，订单服务调用库存服务扣减库存。

2. 测试步骤：

- ✓ 用户登录（验证身份服务）。
- ✓ 提交订单（验证订单服务与数据库）。
- ✓ 检查库存是否减少（验证库存服务接口）。
- ✓ 模拟支付成功回调（验证第三方支付集成）。

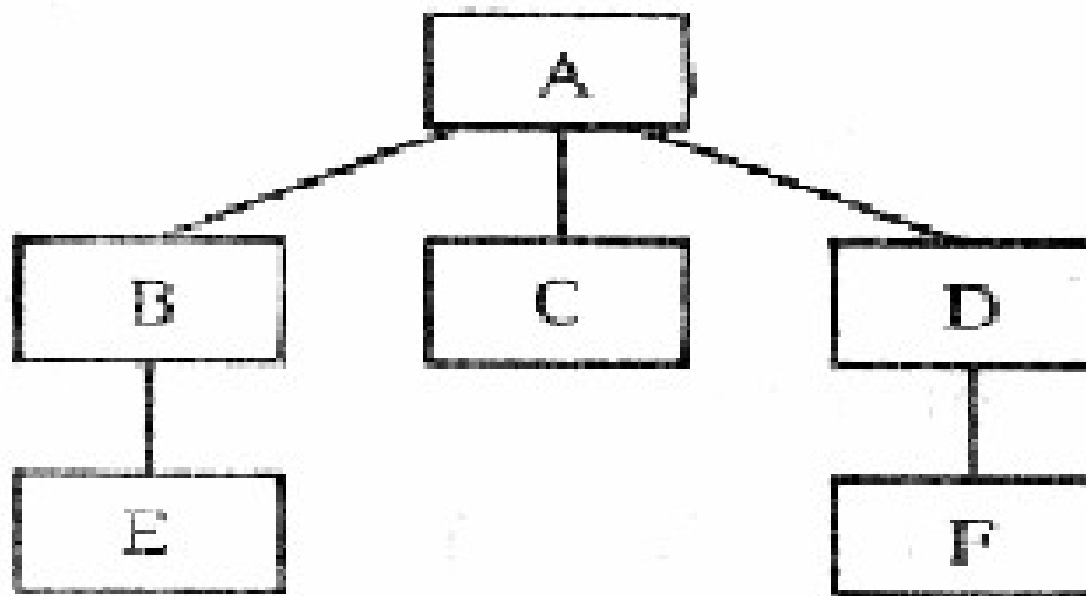
3. 可能暴露的问题：

- ✓ 订单服务未正确处理库存不足的异常。
- ✓ 付回调后订单状态未更新。

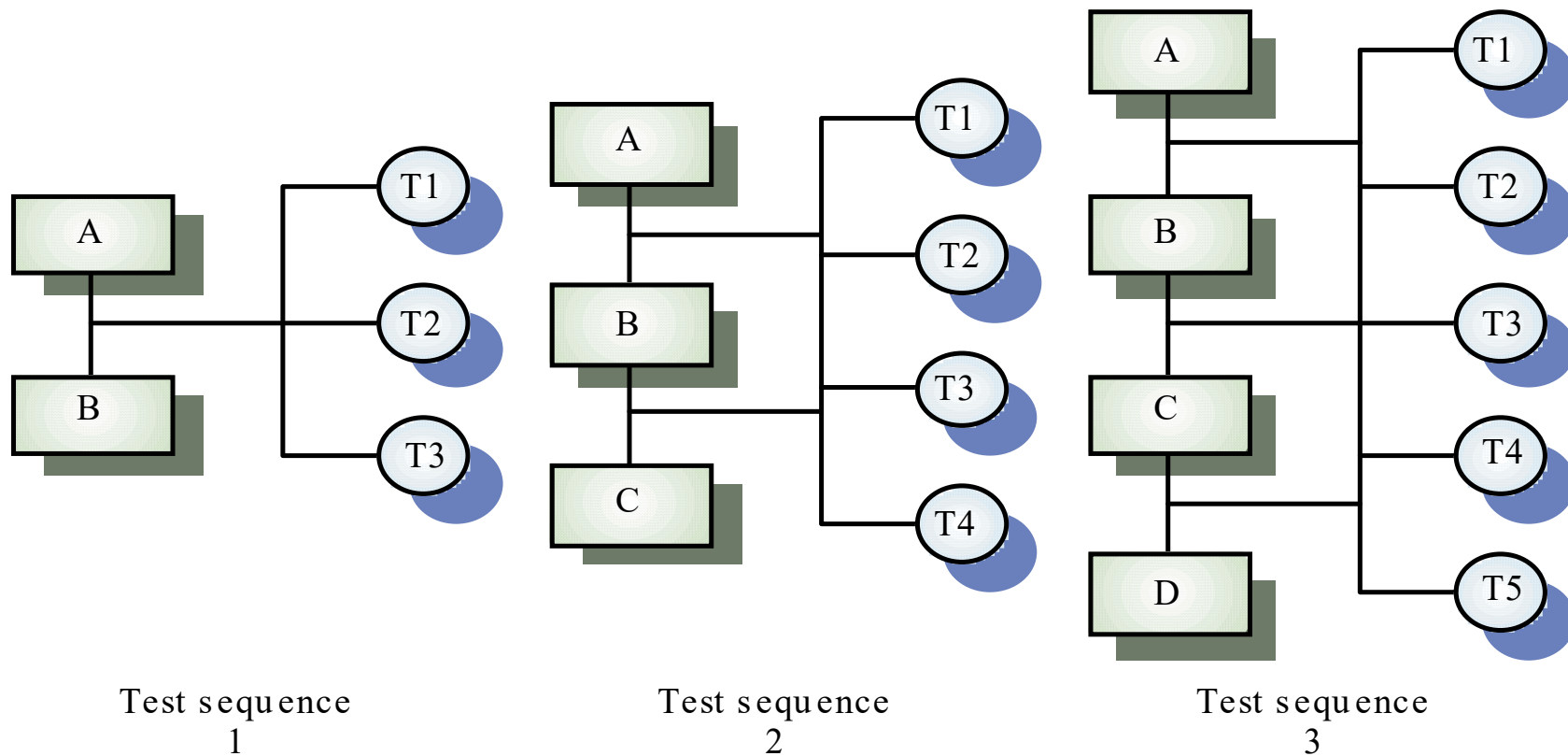
集成测试的方法

- ❑ **非增式集成**（No-Incremental Integration）：非增式集成也被称为大爆炸集成（Big Bang Integration）。采用一步到位的方法来构造测试对象，即对所有模块进行个别的单元测试后，按程序结构图将各模块联接起来，把联接后的程序当作一个整体进行测试。
- ❑ **增式集成**（Incremental Integration）：把下一个要测试的模块同已经测试好的模块结合起来进行测试，一次增加一个要测试模块。

非增式集成



增式集成

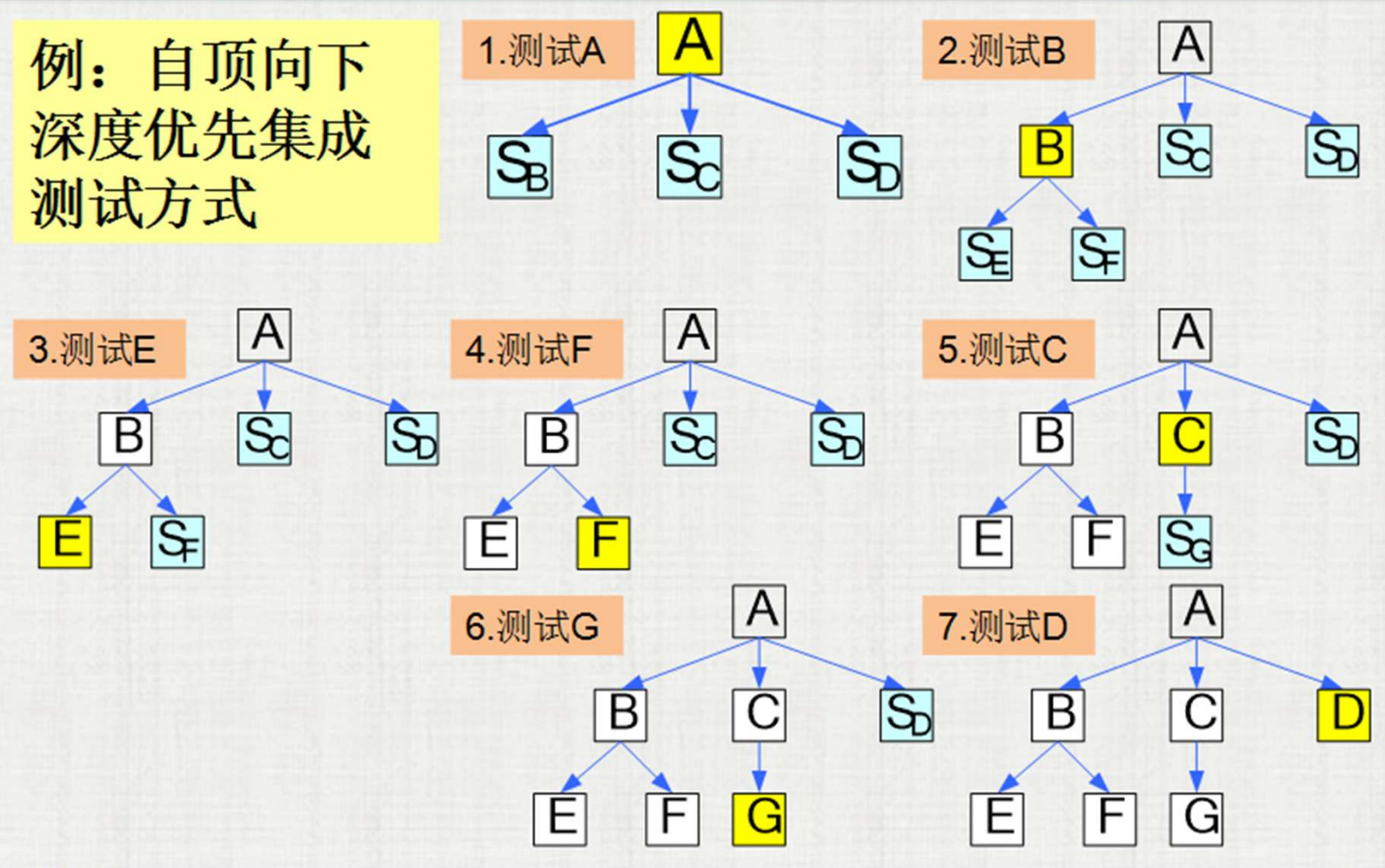


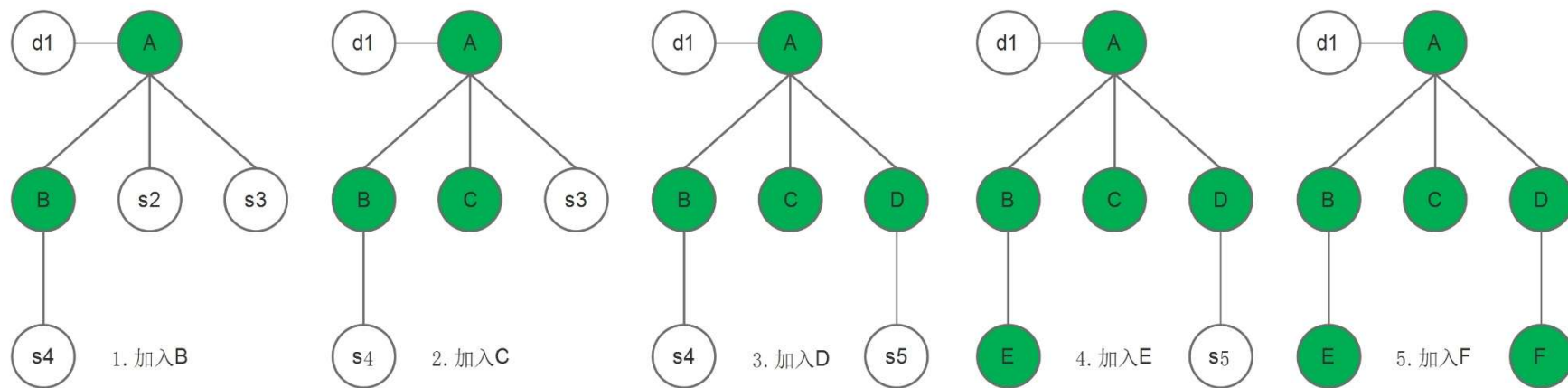
增式集成

- 自顶向下集成
- 自底向上集成

自顶向下增式集成

例：自顶向下
深度优先集成
测试方式





自顶向下增式集成.....

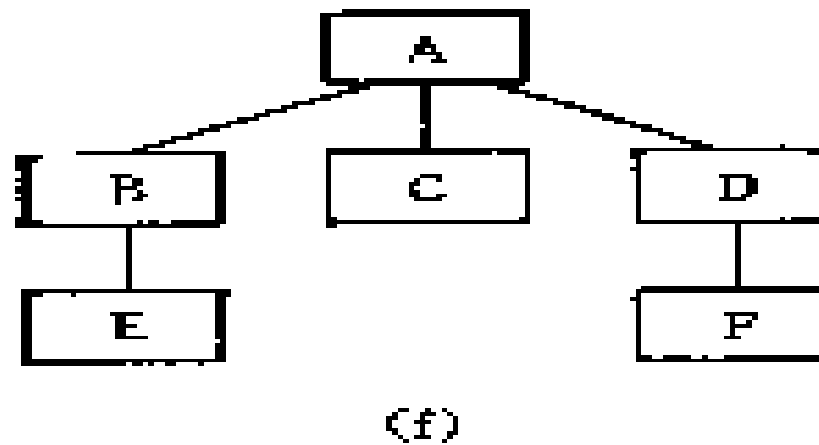
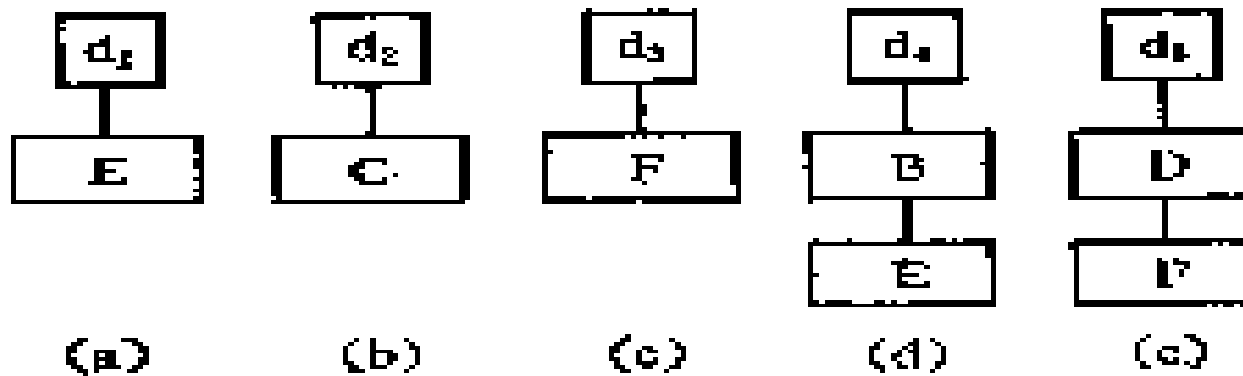
□集成步骤：

1. 主控模块作为测试驱动，所有与主控模块直接相连的模块作为桩模块；
2. 根据集成的方式（深度或广度），每次用一个替换从属的桩模块；
3. 在每个模块被集成时，都必须已经进行了单元测试；
4. 进行回归测试以确定集成新模块后没有引入错误

□上述过程从第2步重复进行，直到整个系统结构被集成完成。

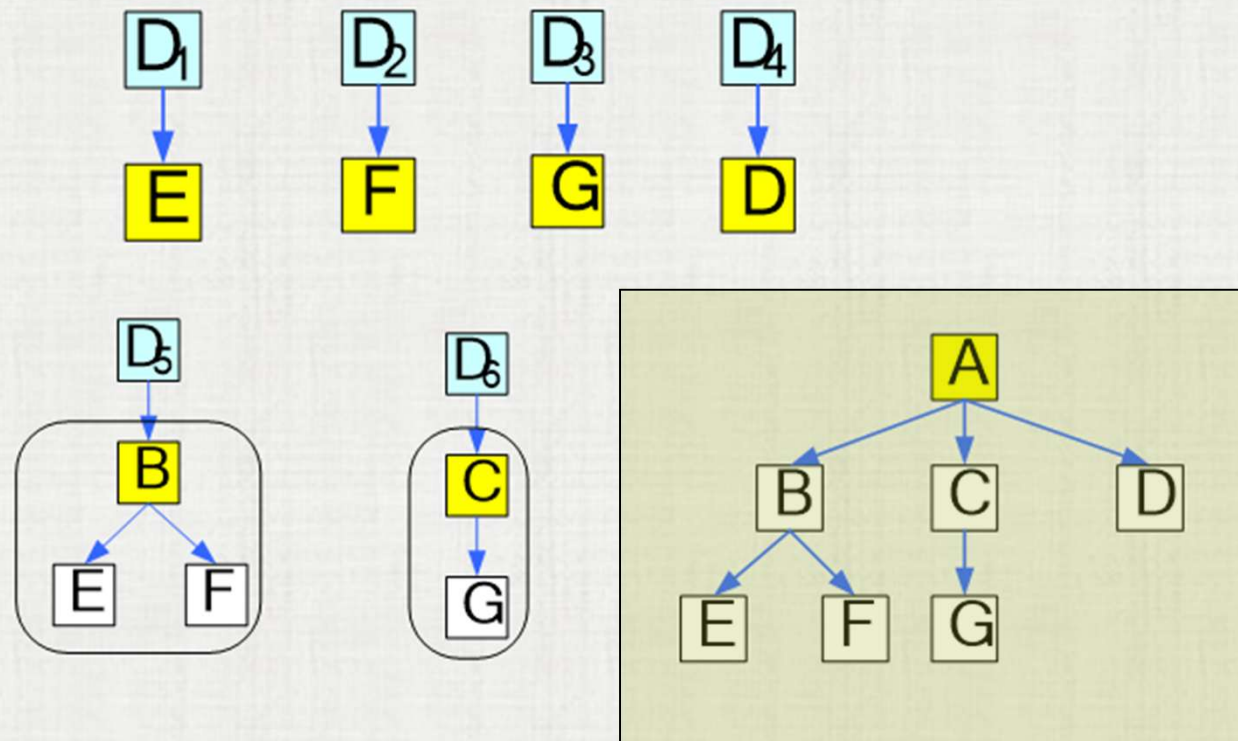
□桩模块（Stub）是指在软件测试中用于模拟被测试模块所调用的其他模块的虚拟模块。它的主要作用是代替被测模块的实际依赖模块，以解决被测模块在集成测试中因依赖模块未完成或未测试而无法进行测试的问题

自底向上增式测试



自底向上增式测试.....

- 例：自底向上广度优先集成测试方式



自底向上增式测试.....

□ 工作程序：

- 集成从最底层的模块开始，组合成一个构件，用以完成指定的软件子功能
- 编制**驱动模块**，协调测试用例的输入与输出；
- 测试集成后的构件
- 按**程序结构**向上集成测试后的构件，同时除掉驱动模块

驱动模块（driver）在软件测试中是用来模拟被测试模块的**上一级模块**，相当于被测模块的主程序。它的主要作用是**接收数据，将相关数据传送给被测模块，启动被测模块，并打印出相应的结果**。驱动模块的主要目的是为了访问类库的属性和方法，检测类库的功能是否正确

两种集成方法的比较

	非增式集成	增式集成
工作量	工作量大	工作量小
接口错误	发现错误较晚	发现错误早
错误定位	错误定位难	错误定位易
测试程度	测试不彻底	测试彻底
测试的并行性	并行性好	并行性差

两种增式集成测试方法的比较

	优点	缺点
自顶向下集成	可以自然地做到逐步求精，一开始便能让测试者看到系统的框架	■ 需要提供桩模块 ■ 在输入/输出模块接入系统以前，在桩模块中表示测试数据有一定困难 ■ 由于桩模块不能模拟数据，如果模块间的数据流不能构成有向的非环状图，一些模块的测试数据难于生成； ■ 观察和解释测试输出往往也是困难的
自底向上集成	■ 由于驱动模块模拟了所有调用参数，即使数据流并未构成有向的非环状图，生成测试数据也没有困难 ■ 特别适合于关键模块在结构图底部的情况	■ 直到最后一个模块被加进去之后才能看到整个程序（系统）的框架 ■ 只有到测试过程的后期才能发现时序问题和资源竞争问题

1.4 系统测试

系统测试（System Testing）是软件测试的重要阶段，旨在**验证完整的软件系统是否符合需求规格说明书（SRS）中的功能和非功能要求**。它关注的是整个系统作为一个统一体的行为，而非单个模块或组件，通常在集成测试之后、用户验收测试（UAT）之前执行。

系统测试的一般情况是：

- ✓所有的集成测试完成
- ✓软件系统之间的联合测试
- ✓软件、硬件等之间的联合测试
- ✓模拟真实运行环境的测试

核心概念

1. 测试对象：

- ✓ 完整的、已集成的软件系统（包括所有模块、服务和依赖项）。
- ✓ 部署在真实或接近真实的环境中（如测试服务器、预发布环境）。

2. 测试范围：

- ✓ 功能性需求：验证系统功能是否满足用户需求（如登录、支付、搜索）。
- ✓ 非功能性需求：性能、安全性、兼容性、可靠性等（如支持1000并发用户、响应时间小于2秒）。

3. 测试视角：

- ✓ 黑盒测试：不关注内部代码逻辑，仅通过输入和输出来验证系统行为。
- ✓ **端到端测试**：模拟真实用户操作流程（如从注册到下单的完整业务流程）。

端到端测试（End-To-End Testing，简称E2E测试）是一种从头到尾测试整个软件产品的方法，以确保应用程序流程按预期运行。它模拟真实用户场景，验证被测系统及其组件的集成和数据完整性，主要从最终用户的体验进行测试。

核心目的

1. 验证系统完整性：

- ✓ 确保所有功能模块协同工作，满足整体业务目标。
- ✓ 发现集成测试未覆盖的场景（如跨模块的数据流、全局配置问题）。

2. 评估非功能性指标：

- ✓ 性能：压力测试、负载测试、稳定性测试。
- ✓ 安全性：渗透测试、漏洞扫描。
- ✓ 兼容性：不同浏览器、操作系统、设备上的表现。

3. 为验收测试铺路：

- ✓ 确保系统达到可交付状态，减少用户验收测试（UAT）中的问题。

核心目的

1. 验证系统完整性：

- ✓ 确保所有功能模块协同工作，满足整体业务目标。
- ✓ 发现集成测试未覆盖的场景（如跨模块的数据流、全局配置问题）。

2. 评估非功能性指标：

- ✓ 性能：压力测试、负载测试、稳定性测试。
- ✓ 安全性：渗透测试、漏洞扫描。
- ✓ 兼容性：不同浏览器、操作系统、设备上的表现。

3. 为验收测试铺路：

- ✓ 确保系统达到可交付状态，减少用户验收测试（UAT）中的问题。

实施步骤

1. 测试需求分析：

- ✓ 基于需求文档明确测试范围和优先级。 例如：电商系统需支持用户注册、商品搜索、支付流程。

2. 设计测试用例：

- ✓ 覆盖正常场景、异常场景和边界条件。 示例：
 - 正常：用户使用正确密码登录成功。
 - 异常：输入错误密码3次后账户锁定。
 - 边界：搜索关键词长度为1000字符时的处理。

3. 准备测试环境：

- ✓ 搭建与开发环境一致的硬件、网络、数据库配置。
- ✓ 使用工具生成测试数据（如Mockaroo生成用户数据）。

4. 执行测试：

- ✓ 手动测试：探索性测试复杂业务场景。
- ✓ 自动化测试：使用工具（如Selenium、Cypress）执行重复流程

5. 缺陷管理与回归：

- ✓ 记录缺陷并跟踪修复（工具：Jira、Bugzilla）。
- ✓ 修复后重新运行相关测试用例，确保问题解决且未引入新缺陷。

典型测试类型

测试类型	描述
功能测试	验证系统功能是否符合需求（如支付成功、订单状态更新）。
性能测试	评估系统在高负载下的表现（如响应时间、吞吐量）。
安全测试	检测漏洞（如SQL注入、XSS攻击）。
兼容性测试	验证不同浏览器、操作系统、设备的兼容性（如Chrome、iOS、Android）。
可靠性测试	检查系统在长时间运行或异常条件下的稳定性（如内存泄漏、服务崩溃恢复）。
安装/卸载测试	确保软件安装、升级、卸载过程无误（尤其针对桌面或移动应用）。

工具与框架：

功能测试：

- ✓ Web: Selenium、Cypress、Playwright
- ✓ 移动端: Appium、Espresso（Android）、XCUITest（iOS）

性能测试：

- ✓ 负载测试: JMeter、LoadRunner、Gatling
- ✓ 压力测试: Locust、k6

安全测试： - OWASP ZAP、Burp Suite、Nessus

兼容性测试： - BrowserStack、Sauce Labs、LambdaTest

示例：电商系统系统测试

1. 功能测试：

- ✓ 用户从搜索商品→加入购物车→支付→查看订单状态，整个流程是否畅通。
- ✓ 优惠券叠加使用规则是否正确（如满减券与折扣券同时生效）。

2. 性能测试：

- ✓ 模拟“双11”期间1万用户同时抢购，检查系统是否崩溃或响应延迟。

3. 安全测试：

- ✓ 尝试在支付页面注入恶意脚本，验证系统是否拦截非法输入。

4. 兼容性测试：

- ✓ 在iPhone、Android手机、PC端的Chrome/Safari浏览器上测试页面渲染一致性。

1.5 验收测试

验收测试（Acceptance Testing）是软件测试的最后阶段，旨在验证系统是否满足用户需求、业务目标和合同条款，决定软件是否可交付给客户或上线使用。它从最终用户或业务方的视角出发，确保软件真正解决了实际问题，而非仅通过技术验证。

- ✓ V模型中测试的最后一道工序
- ✓ 用户在场或者直接测试
- ✓ 用户可能自定义测试用例

1.6 α 测试和 β 测试

通常由用户或其他人（非开发人员和测试人员）来完成

- α 测试是在开发即将完成时对应用进行的测试，此时仍然允许对设计作微小的变动。
- α 测试是由用户在开发环境下进行的测试,也可以是开发机构内部的用户在模拟实际操作环境下进行的测试。开发者坐在用户旁边,这是在开发者受控的环境下进行的测试。由开发者随时记录下错误情况和使用中的问题。

1.6 α 测试和 β 测试...

- β 测试是一种验收测试，**是一种软件发布之前的测试**。验收测试一般根据产品规格说明书严格检查产品, 逐行逐字地对照说明书上对软件产品所做出的各方面要求, 确保所开发的软件产品符合用户的各项要求。
- β 测试是软件的多个用户**在一个或多个用户的实际使用环境下**进行的测试. 开发者通常不在测试现场, 不能由程序员或测试员完成。

1.7 回归测试

回归测试（Regression Testing）是软件开发过程中确保代码修改不会破坏现有功能的一种测试方法。其核心目的是验证在添加新功能、修复缺陷或优化代码后，系统中已有的功能仍能正常工作。

为什么需要回归测试？

当软件被修改时，新的代码可能无意中引入错误，影响之前正常的功能。

例如：修复一个登录功能的 Bug 后，可能导致注册功能失效。回归测试能及时发现这类问题。

实际案例：假设电商网站修复了“支付失败”的 Bug，回归测试需验证：

- ✓ 支付功能是否修复。
- ✓ 购物车、订单生成、库存扣减等关联功能是否正常。
- ✓ 其他核心功能（如登录、搜索）是否受影响。

1.7 回归测试...

□ 目标

- ① 修改的或增加的部分是正确的
- ② 没有引起其他部分产生错误

□ 应用

- ① 增量开发
- ② 版本控制
- ③ 软件维护

1.7 回归测试.....

- 方法举例:

- 全部再测试(Retest All)
- 按风险再测试(Retest Risky Use Case)

rerun by risk

- 按频率再测试(Retest by Profile):

rerun by allocating time in proportion to operational profile

- 按修改再测试(Retest Changed Segments):

rerun by comparing code changes

- 按依赖再测试(Retest Within Firewall)

一个典型的回归测试过程

- 回归测试一般遵循下述基本过程：
 - (1)识别出软件中被修改的部分。
 - (2)从原基线测试用例库T中，排除所有不再适用的测试用例，确定那些对新的软件版本依然有效的测试用例，其结果是建立一个新的基线测试用例库T0。
 - (3)依据一定的策略从T0中**选择**测试用例测试被修改的软件
 - (4)如果必要，**生成**新的测试用例集T1，用于测试T0无法充分测试的软件部分。
 - (5)用T1执行修改后的软件。
- 第(2)和第(3)步测试验证修改是否破坏了现有的功能。
- 第(4)和第(5)步测试验证修改工作本身。

2 测试过程(面向测试任务)

1. 测试需求描述
2. 测试计划
3. 测试设计
4. 测试开发
5. 测试执行
6. 测试结果与预期结果比较
7. 测试充分性评估
8. 测试报告生成

测试需求

测试计划 标记测试条件（确定测试什么）和测试的优先级

测试设计 设计测试用例（确定怎么测试）

测试开发 测试开发（设计脚本、数据等）

测试执行 执行测试用例

测试比较 将测试结果与期望结果进行比较

测试评估 覆盖度评估

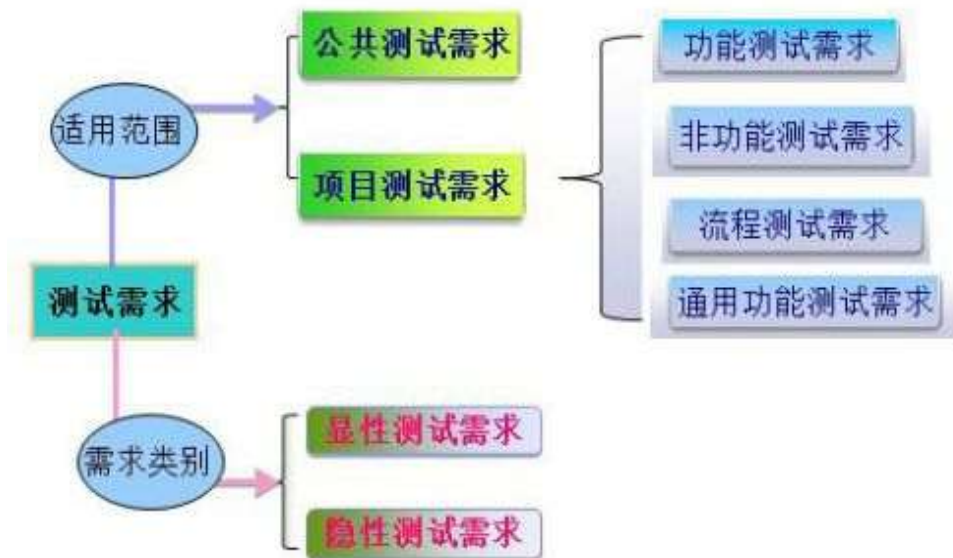
测试报告



2.1 测试需求

- 软件测试需求是测试活动的基础，它定义了"测什么"的问题。
- 我们在测试活动中，首先需要明确测试需求(What)，才能决定怎么测(How)，测试时间(When)，需要多少人(Who)，测试的环境是什么(Where)，测试中需要的技能、工具以及相应的背景知识，测试中可能遇到的风险等，以上所有的内容结合起来就构成了测试计划的基本要素。

测试需求是从软件需求(功能和非功能需求)中提炼出来的、可测试的、明确的验证要求。它来源于用户需求、系统需求和设计文档；它是编写测试用例的依据；它明确了测试范围和测试目标。



测试需求描述例子

测试需求的表达方式:

- ✓ 测试需求列表
- ✓ 测试需求跟踪矩阵(RTM)

例子：在互联网无处不在的今天，IP地址有着至关重要的作用，如果网络IP不正确，将无法访问网站。而本软件可以检测IP地址是否有效。

表 2 功能测试需求列表

测试需求编号	测试需求名称	测试需求描述	前置条件	预期结果	类型	优先级	评审状态
001	正确输入	输入规范的IP地址,测试输出是否正确	输入规范IP地址	输出判断为正确IP地址	功能测试需求	核心	
002	错误输入	输入无效IP地址,测试输出是否正确	输入无效IP地址	判断为无效IP	功能测试需求	核心	
003	非法操作	进行非法输入或动作,测试是否正确警告	非法输入或动作	警告非法输入或动作	功能测试需求	重要	

2.2 计划

2.2.1 测试目标

2.2.2 测试范围

2.2.3 测试项目

2.2.4 测试方法/策略/手段

2.2.5 需要工具

2.2.6 测试资源

2.2.7 交付/产出物件

测试计划是一份详细描述测试目标、范围、方法、资源、进度和交付物的文档，用于协调和组织测试活动。它是测试过程的蓝图，通常由测试经理或测试负责人编写。

2.1.1. 测试目标

□ 单元测试：Unit Testing

□ 目标：

- 检验程序最小单元有无错误
 - 接口、数据结构、边界、覆盖、逻辑
- 检验单元编码与设计是否吻合

□ 集成测试：Integration Testing

□ 目标：

- 检验组成系统的模块接口有无错误
- 代码实现的系统设计与需求定义是否吻合

测试目标...

□ 系统测试：System Testing

□ 目标：

- 检验组成整个系统的代码、以及系统的软硬件配合有无错误
- 代码实现的系统与用户需求是否吻合
- 检验系统的各种文档等是否完整、有效
- 模拟验收测试的要求，检查系统是否符合用户的验收标准

测试目标...

□ 验收测试：Acceptance Testing

□ 目标：

- 使客户验收签字
- 系统是否符合事先约定的验收标准

□ 回归测试：Regression Testing

□ 目标：

- 验证程序修改或者版本更新以后，以前正确的功能和其他指标仍旧正确。

2.1.2. 测试范围

- 功能测试

- ✓ 哪些模块需要做功能测试

- 接口测试

- ✓ 哪些子系统的接口需要测试

- 性能测试，负载测试等

- ✓ 哪些功能要做性能测试

2. 1. 3. 测试项目 (items)

- 数据和数据库集成测试
 - ✓ 验证×记录并发访问...
- 功能测试
 - ✓ 依据×文档，×. ×节的“特征”
- GUI 测试
 - ✓ 依据规格说明书的相关叙述
- 性能测试
 - ✓ 测试访问某功能的响应时间

测试项目...

- 安全测试
- 压力测试
- 负载测试
- 容量测试
- 失效/恢复测试
- 安装测试
- 配置测试
- ...

2. 1. 4. 测试方法/策略/手段

- Code coverage strategies, e.g.
 - Decision coverage
 - Data-flow testing (Defines -> Uses)
- Specification-based testing, e.g.
 - Equivalence partitioning
 - Boundary-value analysis
 - Combination strategies
- State-based testing

2.1.5. 测试工具

- 测试管理工具
- 测试设计工具
- 缺陷跟踪工具
- 数据管理工具
- 功能测试工具
- 性能测试工具
- 其他工具

工具选择应考虑项目需求、技术栈、团队技能和预算等因素。现代测试实践中，往往需要组合使用多种工具构建完整的测试工具链。

软件测试工具可以按照测试活动的不同阶段和用途进行系统分类。以下是全面的测试工具类型划分及代表工具：

工具类型	作用	典型工具举例
测试管理工具	用于规划、组织和控制测试过程	TestRail: 测试用例管理和执行跟踪; JIRA+Zephyr: 缺陷管理与测试流程结合; qTest: 企业级测试管理平台; PractiTest: 端到端测试管理解决方案; TestLink: 开源测试管理工具。
测试设计工具	辅助测试用例设计和需求分析	MindMap工具 (XMind/MindManager): 测试场景可视化; PICT: 微软开发的组合测试用例生成工具; TestOptimal: 基于模型的测试设计工具; Tcases: 自动生成边界值测试用例; ReqView: 需求管理与测试用例关联工具。
功能测试工具	验证软件功能正确性	Selenium: Web UI自动化测试框架; Appium: 跨平台移动应用测试; Cypress: 现代Web测试框架; Playwright: 跨浏览器自动化测试; Robot Framework: 关键字驱动测试框架。
性能测试工具	评估系统性能指标	Jmeter: 开源负载测试工具; LoadRunner: 企业级性能测试解决方案; Gatling: 高性能负载测试工具; Locust: Python编写的分布式压测工具; k6: 开发者友好的性能测试工具。
安全测试工具	识别系统安全漏洞	OWASP ZAP: Web应用安全扫描; Burp Suite: 渗透测试平台; Nessus: 漏洞扫描工具; Metasploit: 渗透测试框架; SonarQube: 代码安全与质量分析。
API测试工具	验证接口功能与性能	Postman: API开发与测试平台; SoapUI: SOAP/REST测试工具; Insomnia: API调试工具; Karate: API自动化测试框架; Rest-Assured: Java DSL for REST测试。
移动测试工具	专门用于移动应用测试	Espresso: Android UI测试框架; XCUITest: iOS UI测试框架; AWS Device Farm: 云端真机测试; Firebase Test Lab: Google移动测试服务; Appium: 跨平台移动自动化。
测试自动化框架	提供自动化测试基础架构	TestNG: Java测试框架; Junit: Java单元测试框架; pytest: Python测试框架; Mocha/Jest: JavaScript测试框架; Cucumber: BDD测试框架。
持续测试工具	支持CI/CD流水线	Jenkins: 持续集成服务器; GitLab CI/CD: 集成测试与部署; CircleCI: 云端CI/CD平台; Travis CI: - GitHub集成测试; Azure DevOps: 微软测试管道。
专项测试工具	满足特定测试需求	Squish: GUI测试工具; Eggplant: AI驱动的工具; TestComplete: 商业自动化测试工具; Ranorex: 桌面和移动应用测试; Applintools: 视觉回归测试。
测试数据管理工具	生成和管理测试数据	Mockaroo: 测试数据生成器; DataFactory: 测试数据生成工具; DBUnit: 数据库测试框架; Faker: 多语言测试数据生成库; TDM (Test Data Manager): 企业级测试数据管理。
测试覆盖率工具	衡量代码测试完整性	JaCoCo: Java代码覆盖率; Cobertura: Java代码覆盖率; Coverage.py: Python代码覆盖率; Istanbul: JavaScript代码覆盖率; OpenCover: .NET代码覆盖率。
缺陷管理工具	跟踪和管理软件缺陷	Bugzilla: 开源缺陷跟踪系统; MantisBT: 轻量级缺陷跟踪; Redmine: 项目管理与缺陷跟踪; HP ALM/Quality Center: 企业级质量管理; TFS/Azure DevOps: 微软缺陷管理。

2. 1. 6. 测试资源

□ 角色

- ✓角色名称： 经理，设计人，测试人等
- ✓角色安排：
- ✓责任

□ 系统资源

- ✓软件
- ✓硬件

2.1.7 交付/产出物件

- 测试计划
- 测试环境
- 测试包
- 测试日志
- 缺陷报告

2.3 设计

- 测试用例设计：确认和详细描述测试用例
- 测试脚本设计：确认和设计测试脚本
- 覆盖准则设计：评估测试覆盖

2.3.1 测试用例设计

广义地分为两类：

- ✚ 黑盒测试：使用单元接口和功能描述，不需了解被测单元的内部结构
- ✚ 白盒测试：使用被测单元内部如何工作的信息
- ✚ 其他技术：不属于以上 2 类

Black box (functional)	White box (structural)	Other
Specification derived tests	Branch testing	Error guessing
Equivalence partitioning	Condition testing	
Boundary value analysis	Data definition-use testing	
State-transition testing	Internal boundary value testing	

测试设计最重要的因素是经验和常识。测试设计者不应该让某种测试技术阻碍经验和常识的运用。

测试用例模板和例子

项目/软件	技术出口合同网络申领系统（企业端）	程序版本	1.0.25		
功能模块名	Login	编制人	xxx		
用例编号-	TC-TEP_Login_1	编制时间	2002.10.12		
相关的用例	无				
功能特性	用户身份验证				
测试目的	验证是否输入合法的信息，允许合法登陆，阻止非法登陆				
预置条件	无	特殊规程说明	如数据库访问权限		
参考信息	需求说明中关于“登陆”的说明				
测试数据					
操作步骤	操作描述	数据	期望结果	实际结果	测试状态 (P/F)
1	输入用户名称，按“登陆”按钮。	用户名=yiyh，密码为空	显示警告信息“请输入用户名和密码！”		
2	输入密码，按“登陆”按钮。	用户名为空，密码=1	显示警告信息“请输入用户名和密码！”		

[权限管理系统]项目测试用例

系统模块	功能点	用例编号	用例说明	前置条件	输入	预期结果	测试结果	失败原因
	2.1 增加用户	2.1.1	新增用户动作	系统管理员已登录系统	系统管理员在用户管理页面中单击“新增用户”按钮	成功进入到“新增用户”页面		
		2.1.2	必填项	系统管理员已登录系统，并进入到“新增用户”页面	令某项必填数据为空，其余数据正常填写，提交（例如，将用户名置空，提交）	提示“请输入×××”（例如：请输入用户名）		
		2.1.3	数据有效性	系统管理员已登录系统，并进入到“新增用户”页面	在本框中输入无效数据，提交（例如，输入非法E-mail：cxw521816.hxf999.com, 提交）	提示“请输入合法的××”（例如：请输入合法的E-mail地址）		
		2.1.4	全部置空	系统管理员已登录系统，并进入到“新增用户”页面	令所有文本框为空，提交	提示“请输入用户名”		
		2.1.5	输入范围	系统管理员已登录系统，并进入到“新增用户”页面	在文本框中输入长度为999的文本提交（例如，在用户名文本框中输入长度为999的文本）	输入框本身应有输入范围限制，多出“最大输入位数”部分应自动舍弃		
		2.1.6	提交新增用户信息	系统管理员已登录系统	在“新增用户”页面各项文本框中输入有效数据，提交	系统提示“新增用户成功”，成功添加系统用户		
			删除用户	系统管理员已登录系统	系统管理员单击“删除”按钮	弹出系统提示框“是否真的		

2.3.2 测试脚本设计

- 测试脚本（Test Script）是一组自动化执行的指令或代码，用于模拟用户操作或验证软件功能。它替代了手动测试的重复步骤，通过编程语言或工具实现测试用例的自动运行，提高效率并减少人为错误。
- 测试脚本的核心作用
 - 自动化重复操作：例如登录、表单提交、数据验证等高频操作。
 - 精准验证结果：通过断言（Assert）自动判断测试是否通过。
 - 支持持续集成（CI/CD）：与 Jenkins、GitLab 等工具集成，实现代码提交后自动测试。

测试脚本的常见类型与示例：UI 自动化测试脚本（Web/App）

-场景：测试用户界面功能，如网页登录、按钮点击。

-工具：Selenium（Web）、Appium（移动端）。

-示例（Python + Selenium）：

```
python
from selenium import webdriver

# 打开浏览器并访问网页
driver = webdriver.Chrome()
driver.get("https://example.com/login")

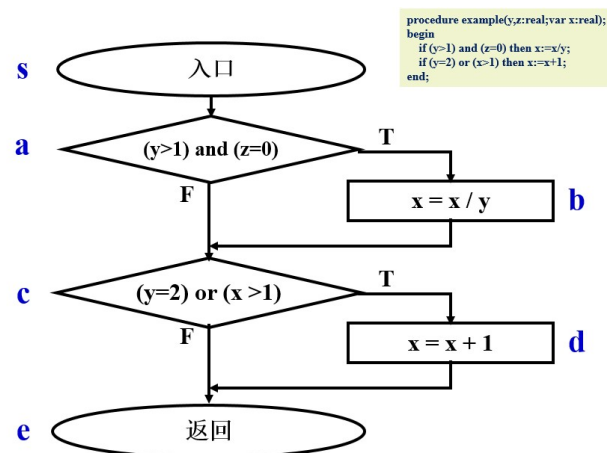
# 输入用户名密码并点击登录
driver.find_element("id", "username").send_keys("test_user")
driver.find_element("id", "password").send_keys("123456")
driver.find_element("id", "login_btn").click()

# 验证登录后页面标题
assert driver.title == "用户主页", "登录失败！"
driver.quit()
```

2.3.3 测试覆盖准则设计

- 覆盖率
- 控制流覆盖率
- 数据流覆盖率
- 分支覆盖率
- 路径覆盖率
-

常用覆盖准则		
1	语句覆盖	保证程序中的 每条语句 都执行一遍
2	判定覆盖	保证 每个判断 取True和False至少一次
3	条件覆盖	保证每个 判断中的每个条件 的取值至少满足一次
4	判定条件覆盖	保证 每个条件和由条件组成的判断 的取值...
5	条件组合覆盖	保证 每个条件的取值组合 至少出现一次...
6	路径覆盖	覆盖程序中所有可能路径



2.4 开发

- 搭建测试环境
- 录制或编写测试脚本
- 开发测试驱动模块(drivers)
- 开发桩模块 (stubs)
- 建立外部数据集

如何搭建测试环境？

- 搭建测试环境是软件测试的关键步骤，其目的是模拟真实运行场景，确保测试结果的准确性和可靠性。以下是详细的步骤和示例：
- 在测试环境搭建之前需要明确测试环境的目标
 - 1. 确定测试类型：功能测试、性能测试、安全测试等。
 - 2. 匹配需求：
 - ✓ 硬件：服务器配置（CPU、内存、存储）。
 - ✓ 软件：操作系统、数据库、中间件、依赖库版本。
 - ✓ 数据：测试数据集（需脱敏）。
 - ✓ 网络：内网/外网访问权限、防火墙规则。

建立测试环境的核心是模拟真实场景并保持可控性。通过容器化、自动化工具和严格的数据管理，可显著提高测试效率和可靠性。务必确保测试环境与生产环境高度一致，避免“测试通过，上线崩溃”的尴尬局面。

搭建测试环境的步骤

1. 准备硬件与软件资源：
 - ✓ 物理机/虚拟机/容器
2. 安装依赖软件：
 - ✓ 操作系统、中间件、开发语言环境
3. 配置服务与网络：
 - ✓ 服务配置、网络隔离
4. 准备测试数据：
 - ✓ 数据来源
5. 自动化部署与验证：
 - ✓ 工具
6. 集成测试工具：
 - ✓ 功能测试、性能测试

实际案例

场景：搭建电商系统的测试环境

1. 硬件：

2台云服务器（4核8G），1台MySQL数据库。

2. 软件：

-前端：Nginx + React 18。
-后端：Spring Boot + Redis。

3. 自动化：

- 使用 Jenkins 拉取代码并构建 Docker 镜像。
- 通过 Selenium 自动化测试下单流程。

驱动程序和桩模块的开发

- **驱动程序和桩模块的作用**：在模块化测试中，当被测模块依赖其他未完成或不可用的组件时，需通过驱动程序和桩模块模拟其行为，实现隔离测试。
- 驱动程序（Driver）：**调用被测模块**（如代替未完成的上层模块）。
- 桩模块（Stub）：**替代被测模块依赖的下层模块**（如模拟数据库或外部接口）。

开发驱动程序的步骤与示例

- 场景：测试一个订单计算模块（OrderCalculator），但调用它的订单管理模块（OrderManager）尚未开发完成。
- 步骤
 1. 定义被测模块的输入输出：1) 输入：用户ID、商品列表；2) 输出：订单总金额。
 2. 编写驱动程序：驱动程序直接调用OrderCalculator，模拟OrderManager的调用逻辑。
- 示例（Java + JUnit）：Java code

```
public class OrderCalculatorDriver {  
    public static void main(String[] args) {  
        OrderCalculator calculator = new OrderCalculator(); // 模拟输入数据  
        String userId = "user_123";  
        List<Product> products = Arrays.asList(  
            new Product("item1", 100.0),  
            new Product("item2", 200.0)        ); // 调用被测模块  
        double total = calculator.calculateTotal(userId, products); // 验证结果  
        System.out.println("订单总额: " + total);  
        assert total == 300.0 : "计算结果错误";  
    }  
}
```

关键点：驱动程序需模拟真实调用参数和流程。可通过单元测试框架（如JUnit）集成，实现自动化验证。

开发桩模块的步骤与示例

- 场景：测试支付服务模块（PaymentService），但它依赖的银行接口（BankAPI）尚未实现或测试环境不可用。

- 步骤

1. 分析依赖接口：BankAPI 提供的方法：boolean transfer(double amount, String account)。
2. 编写桩模块：桩模块模拟BankAPI的响应（如固定返回“转账成功”）。

示例（Python + unittest.mock）：python

```
from unittest.mock import Mock
```

```
# 被测模块：支付服务
```

```
class PaymentService:
```

```
    def __init__(self, bank_api):
```

```
        self.bank_api = bank_api
```

```
    def pay(self, amount, account):
```

```
        return self.bank_api.transfer(amount, account)
```

```
# 开发桩模块
```

```
def test_payment_success():
```

```
    # 1. 创建桩对象，模拟 transfer 方法返回 True
```

```
    stub_bank_api = Mock()
```

```
    stub_bank_api.transfer.return_value = True
```

```
    # 2. 注入桩模块到被测对象
```

```
    payment_service = PaymentService(stub_bank_api)
```

```
    # 3. 调用并验证
```

```
    result = payment_service.pay(100.0, "account_123")
```

```
    assert result is True, "支付应成功"
```

2.5 测试执行和测试结果确认

- 执行测试脚本
- 测试执行情况分析
- 结果验证和确认

软件测试结果验证是确保测试结果准确、可靠的核心环节，其目标是确认测试结果真实反映软件质量，避免误判或遗漏关键问题。例如，在软件测试中，比较测试结果和预期结果是验证软件功能是否正确核心步骤。

软件测试结果确认是确保测试活动有效性和软件质量的关键环节，目的是验证测试的完整性和准确性，并为发布决策提供可靠依据。

手动测试结果对比方法

- 逐项检查法

- 适用场景：业务流程简单、结果可直观观察的测试。

- 步骤：

- 1. 执行测试用例，记录实际结果。
 - 2. 与用例文档中的预期结果逐项比对。
 - 3. 标记差异点并分析原因。

- 示例：

- 测试用例：用户登录功能
 - ✓预期结果：输入正确账号密码后跳转至主页。
 - ✓实际结果：页面提示“密码错误”。
 - 结论：测试失败，需检查密码验证逻辑。

自动化测试结果对比方法

- **断言 (Assertion)**
- 原理：在代码中预设条件，自动判断实际结果是否符合预期。
- 常用断言类型：
 - 等于断言：`assertEqual(actual, expected)`
 - 包含断言：`assertContains(text, substring)`
 - 布尔断言：`assertTrue(condition)`
- 示例 (**Python + pytest**) : **python**

```
def test_add_to_cart():  
    # 添加商品到购物车  
    cart.add_item("product_001", 2)  
    # 断言购物车中商品数量为2  
    assert cart.get_quantity("product_001") == 2, "商品数量不匹配"
```


2.6测试评估和测试报告

- 覆盖准则
- 分析测试用例覆盖
- 分析代码覆盖
- 分析缺陷
- 分析是否达到测试停止、成功标准
- 写测试分析报告

评估

□ 测试用例覆盖

- ✓ Ratio Test Cases Performed = $110/120 = 92\%$
- ✓ Ratio Test Cases Successful = $95/110 = 87\%$

□ 代码覆盖

- ✓ Ratio LOC executed = $94,399 / 102,000 = 93\%$

□ 缺陷分析

- ✓ 缺陷分布（严重性，源代码，时间长度）
- ✓ 缺陷趋势
- ✓ 缺陷状态（记录，提交，修改，测试，关闭）

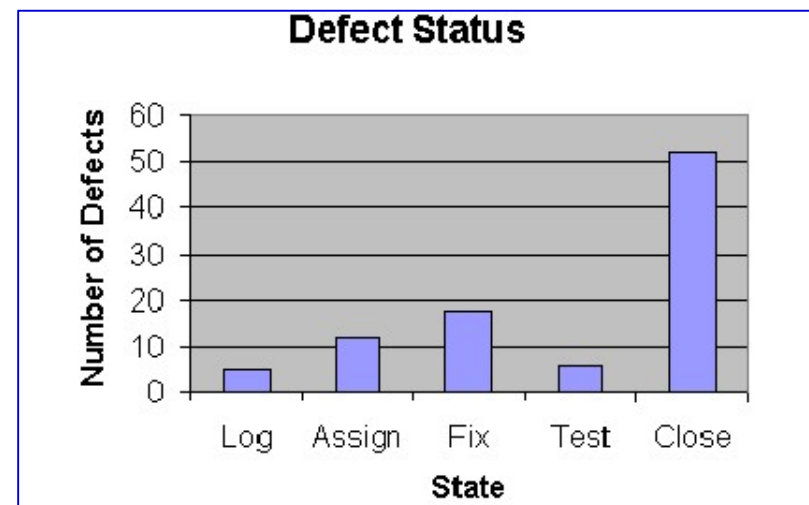
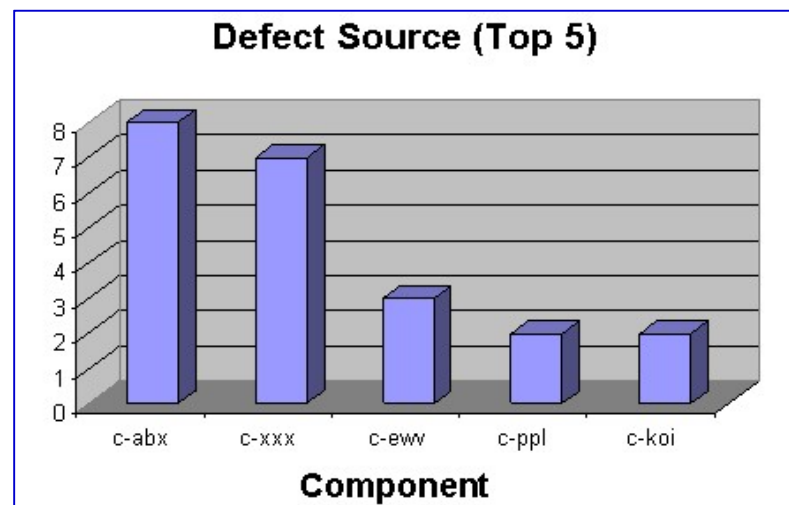
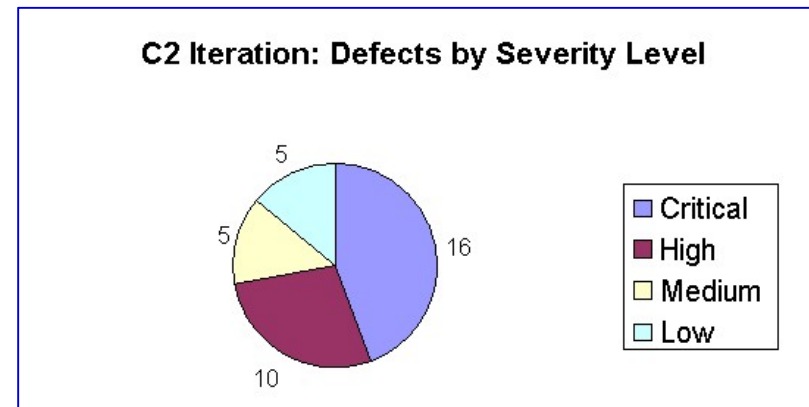
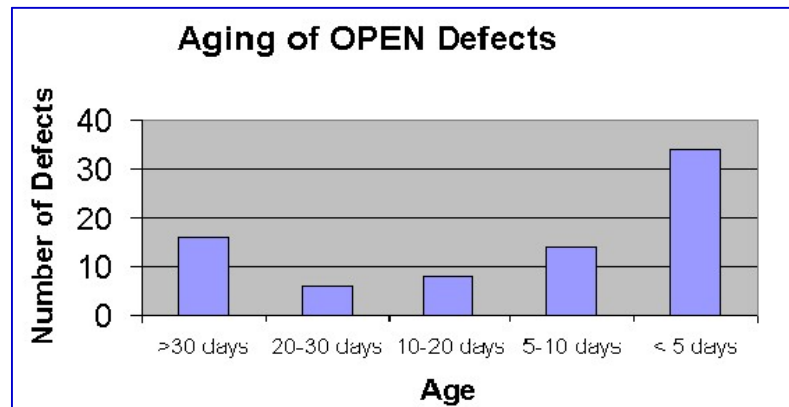
□ 图表

测试报告样例

- 测试报告是记录软件测试过程、结果和结论的正式文档，用于向团队、管理层或客户清晰展示测试活动的执行情况和产品质量。它是测试阶段的最终交付物，帮助决策者判断软件是否达到发布标准。
- 测试报告的核心内容

模块	说明
1. 概述	测试目标、范围、时间、参与人员、被测版本。
2. 测试环境	硬件/软件配置、网络环境、测试工具。
3. 测试执行总结	测试用例总数、通过率、缺陷统计（严重程度分布）、测试覆盖率。
4. 缺陷分析	关键缺陷列表、修复状态、遗留风险。
5. 结论与建议	质量评估、是否达到发布标准、后续改进建议。

Diagram



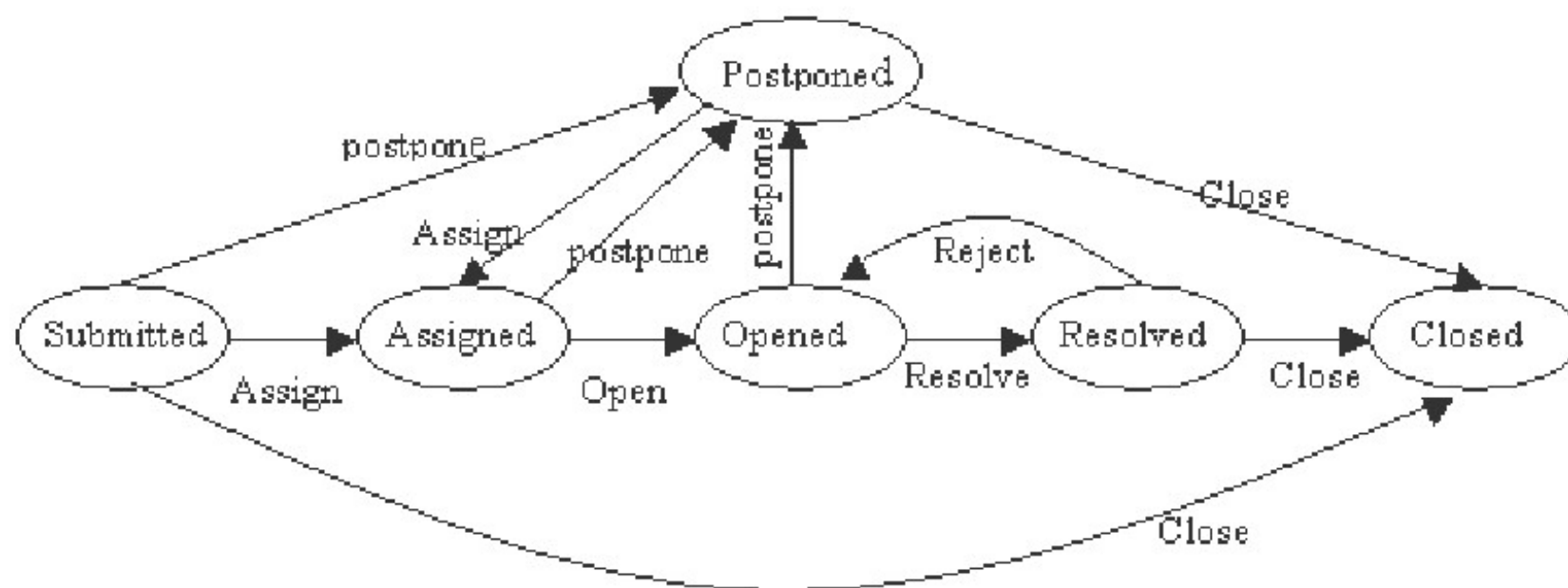
Bug 报告模板

Id	Bug 的唯一标志		
标题	简明扼要地对 Bug 进行概要描述		
产品名称	软件产品的名称		
功能模块名		产品子系统	
产品版本		测试环境	
开发人员		测试人员	
创建时间		Bug 状态	
前提条件	问题来源，引起问题的前提条件		
Bug 严重程度			
问题优先级			
操作步骤			
实际结果			
期望结果			
出现频率			
文字注释和附图			

Bug Report Summary

	Severity 1	Severity 2	Severity 3	Total
Cross P	0	1	10	11
Edit	6	25	68	99
File	27	23	44	94
Fmt Retn	17	147	14	178
Help	0	3	50	53
Install	1	5	20	26
Locate	15	20	32	67
Memory	5	4	10	19
Recognition	6	19	50	75
Scanner	18	7	16	41
Settings	3	7	19	29
View	2	9	33	44
Total	100	270	366	736

Bug跟踪



3 测试方法分类

- ❑ 静态测试和动态测试
- ❑ 黑盒测试和白盒测试
- ❑ 开发者测试、第三方测试、用户测试
- ❑

1、从是否关心内部结构来看

(1) **白盒测试**：又称为结构测试或逻辑驱动测试，是一种按照程序内部逻辑结构和编码结构，设计测试数据并完成测试的一种测试方法。

(2) **黑盒测试**：又称为数据驱动测试，把测试对象当做看不见的黑盒，在完全不考虑程序内部结构和处理过程的情况下，测试者仅依据程序功能的需求规范考虑，确定测试用例和推断测试结果的正确性，它是站在使用软件或程序的角度，从输入数据与输出数据的对应关系出发进行的测试。

(3) **灰盒测试**：是一种综合测试法，它将“黑盒”测试与“白盒”测试结合在一起，是基于程序运行时的外部表现又结合内部逻辑结构来设计用例，执行程序并采集路径执行信息和外部用户接口结果的测试技术。

2、从是否执行代码看

(1) **静态测试**：指不运行被测程序本身，仅通过分析或检查源程序的语法、结构、过程、接口等来检查程序的正确性。

(2) **动态测试**：是指通过运行被测程序，检查运行结果与预期结果的差异，并分析运行效率、正确性和健壮性等性能指标。

3、从开发过程级别看

(1) **单元测试**：又称模块测试，是针对软件设计的最小单位----程序模块或功能模块，进行正确性检验的测试工作。其目的在于检验程序各模块是否存在各种差错，是否能正确地实现了其功能，满足其性能和接口要求。

(2) **集成测试**：又叫组装测试或联合，是**单元测试**的多级扩展，是在**单元测试**的基础上进行的一种有序测试。旨在检验软件单元之间的接口关系，以期通过测试发现各软件单元接口之间存在的问题，最终把经过测试的单元组成符合设计要求的软件。

(3) **系统测试**：是为判断系统是否符合要求而对集成的软、硬件系统进行的测试活动、它是将已经集成好的软件系统，作为基于整个计算机系统的一个元素，与计算机硬件、外设、某些支持软件、人员、数据等其他系统元素结合在一起，在实际运行环境下，对计算机系统进行一系列的组装测试和确认测试。

(4) **验收测试**：检查交付的软件是否满足用户的需求。

在**系统测试**中，对于具体的测试类型有：

- (1) 功能测试：对软件需求规格说明书中的功能需求逐项进行的测试，以验证功能是否满足要求。
- (2) 性能测试：对软件需求规格说明书的功能需求逐项进行的测试，以验证功能是否满足要求。
- (3) 接口测试：对软件需求规格说明中的接口需求逐项进行的测试。
- (4) 人机交互界面测试：对所有人机交互界面提供的操作和显示界面进行的测试，以检验是否满足用户的需求。
- (5) 强度测试：强制软件运行在异常乃至发生故障的情况下（设计的极限状态到超出极限），验证软件可以运行到何种程序的测试。
- (6) 余量测试：对软件是否达到规格说明中要求的余量的测试。
- (7) 安全性测试：检验软件中已存在的安全性、安全保密性措施是否有效的测试。
- (8) 可靠性测试：在真实的或仿真的环境中，为做出软件可靠性估计而对软件进行的功能（其输入覆盖和环境覆盖一般大于普通的功能测试）
- (9) 恢复性测试：对有恢复或重置功能的软件的每一类导致恢复或重置的情况，逐一进行的测试。
- (10) 边界测试：对软件处在边界或端点情况下运行状态的测试。
- (11) 数据处理测试：对完成专门数据处理功能所进行的测试。
- (12) 安装性测试：对安装过程是否符合安装规程的测试，以发现安装过程中的错误。
- (13) 容量测试：检验软件的能力最高能达到什么程度的测试。
- (14) 互操作性测试：为验证不同软件之间的互操作能力而进行的测试。
- (15) 敏感性测试：为发现在有效输入类中可能引起某种不稳定性或不正常处理的某些数据的组合而进行的测试。
- (16) 标准符合性测试：验证软件与相关国家标准或规范（如军用标准、国家标准、行业标准及国际标准）一致性的测试。
- (17) 兼容性测试：验证软件在规定条件下与若干个实体共同使用或实现数据格式转换时能满足有关要求能力的测试。
- (18) 中文本地化测试：验证软件在不降低原有能力的条件下，处理中文能力的测试。

4、从执行过程是否需要人工干预来看

(1) **手工测试**：就是测试人员按照事先为覆盖被测软件需求而编写的测试用例，根据测试大纲中所描述的测试步骤和方法，手工地一个一个地输入执行，包括与被测软件进行交互（如输入测试数据、记录测试结果等），然后观察测试结果，看被测程序是否存在问题，或在执行过程中是否会有一场发生，属于比较原始但是必须执行的一个步骤。

(2) **自动化测试**：实际上是将大量的重复性的测试工作交给计算机去完成，通常是使用自动化测试工具来模拟手动测试步骤，执行用某种程序设计语言编写的过程（全自动测试就是指在自动测试过程中，不需要人工干预，由程序自动完成测试的全过程；半自动测试就是指在自动测试过程中，需要手动输入测试用例或选择测试路径，再由自动测试程序按照人工指定的要求完成自动测试）

5、从测试实施组织看

(1) **开发者测试**：开发人员进行的测试

(2) **用户测试**：用户方进行的测试

(3) **第三方测试**：有别于开发人员或用户进行的测试，由专业的第三方承担的测试，目的是为了保证测试工作的客观性

6、从测试所处的环境看

(1) **α测试**：是由一个用户在开发环境下进行的测试，也可以是公司内部的用户在模拟实际操作环境下进行的测试

(2) **β测试**：是用户公司组织各方面的典型终端用户在日常工作中实际使用贝塔版本，并要求用户报告

静态测试和动态测试

- ▣静态测试的主要特征是在用计算机测试源程序时，计算机并不真正运行被测试的程序，只对被测程序进行特性分析。因此，静态测试常称为“静态分析”，静态分析是对被测程序进行特性分析的一些方法的总称。
- ▣动态测试的主要特征是计算机必须真正运行被测试的程序，通过输入测试用例，对其运行情况(输入/输出的对应关系)进行检查和分析。就是我们通常意义上的测试。

静态测试和动态测试的区别如下：

1、被测部分不同

静态测试是指测试不运行的部分,只是检查和审阅,如规范测试、软件模型测试、文档测试等；动态测试是通常意义上的测试，也就是运行和使用软件。

2、测试方式和方法不同

静态测试通过文档评审、代码阅读等方式测试软件；动态测试通过运行程序的测试软件。

静态测试不用执行程序，它主要采用代码走查、技术评审、代码审查的方法对软件产品进行测试；动态测试主要通过构造测试实例、执行程序、分析程序的输出结果这三种方法来对软件进行测试。

静态测试（又称静态分析）

- 静态分析

- 定义：

- 不实际运行程序，而是通过检查和阅读等手段来发现错误并评估代码质量的软件测试技术。也称为静态测试技术。

- 方法：

- 走查：WalkThrough
 - 审查：Inspection
 - 评审：Review
 - 审计：Auditing

■ 静态分析-走查WalkThrough

● 定义:

- 开发组内部进行的，采用讲解、讨论和模拟运行的方式进行的查找错误的活动。

● 经验:

- 限时
 - 避免跑题
- 参加人员
 - 经验丰富的开发人员
 - 和本模块相关的开发人员
 - 本项目组的新人
- 由本模块的开发者进行讲解、回答问题并记录
 - 不要现场修改
- 检查要点
 - 逻辑错误
 - 代码标准/规范/风格

■ 静态分析-审查Inspection

● 定义：

- 开发组内部进行的，采用讲解、提问并使用Checklist方式进行的查找错误的活动。一般有正式的计划、流程和结果报告。

● 经验：

- 以会议的形式，制定会议目标、流程和规则，结束后要编写报告
- 参加人员
 - 经验丰富的开发人员
 - 和本模块相关的开发人员
 - 本项目组的新人
- 由另外一名开发者进行讲解、其他开发者主要按照Checklist进行提问并填表、本模块开发者回答问题并记录
 - 不要现场修改
- 检查要点
 - 设计需求
 - 代码标准/规范/风格

■ 静态分析-评审review

● 定义：

- 开发组、测试组和相关人员(QA、产品经理等)联合进行的，采用讲解、提问并使用**Checklist**方式进行的查找错误的活动。一般有正式的计划、流程和结果报告。

● 经验：

- 以会议的形式，制定会议目标、流程和规则，结束后要编写报告。相关资料要在会议前下发并阅读。
- 参加人员
 - 经验丰富的开发人员
 - 和本模块相关的开发人员
 - 测试组和相关人员
- 由另外一名开发者进行讲解、其他开发者主要按照**Checklist**进行提问并填表、本模块开发者回答问题并记录
 - 不要现场修改
- 检查要点
 - 设计需求
 - 代码标准/规范/风格
 - 文档的完整性和一致性

黑盒测试和白盒测试

- 黑盒测试（Black—box Testing）又称功能测试、数据驱动测试或基于规格说明的测试，是一种从用户观点出发的测试。
- 白盒测试（White—box Testing）又称结构测试、逻辑驱动测试或基于程序的测试。

黑盒测试

- 被测程序被当作一个黑盒，不考虑程序内部结构和内部特性，测试者只知道该程序输入和输出之间的关系或程序的功能，依靠能够反映这一关系和程序功能的需求规格说明书考虑确定测试用例并推断测试结果的正确性。
- 软件的黑盒测试被用来证实软件功能的正确性和可操作性。

白盒测试

- 它依赖于对程序细节的严密检验，针对特定条件和/与循环集设计测试用例，对软件的逻辑路经进行测试。
- 在程序的不同点检验“程序的状态”以判定其实际情况是否和预期的状态相一致。
- 软件的白盒测试用来分析程序的内部结构

白盒测试.....

- 白盒测试要求对某些程序的结构特性做到一定程度的覆盖，或者说是“基于覆盖的测试”。

最为常见的程序结构覆盖有：

- **语句覆盖**：它要求被测程序的每一可执行语句在测试中尽可能都检验过，这是最弱的逻辑覆盖准则；
- **分支覆盖或判定覆盖**：要求程序中所有判定的分支尽可能得到检验；
- **条件覆盖**：当判定式中含有多个条件时，要求每个条件的取值均得到检验；
- **判定 / 条件覆盖**：同时考虑条件的组合值及判定结果的检验；
- **路径覆盖**：只考虑对程序路径的全面检验。

取得测试覆盖的方法——程序插装

黑盒测试与白盒测试的比较

		黑 盒 测 试	白 盒 测 试
测 试 规 划		根据用户的规格说明，即针对命令、信息、报表等用户界面及体现它们的输入数据与输出数据之间的对应关系，特别是针对功能进行测试。	根据程序的内部结构，比如语句的控制结构，模块间的控制结构以及内部数据结构等进行测试。
特 点	优 点	能站在用户立场上进行测试。	能够对程序内部的特定部位进行覆盖测试。
	缺 点	• 不能测试程序内部特定部位。 • 如果规格说明有误，则无法发现。	• 无法检验程序的外部特性。 • 无法对未实现规格说明的程序内部欠缺部分进行测试。
方 法 举 例		基于图的测试 等价类划分 边值分析 比较测试	语句覆盖 判定覆盖 条件覆盖 判定/条件覆盖 基本路径覆盖 循环覆盖

模拟用户操作的测试方法

- ▣ 基于对用户如何使用被测试软件的了解来进行测试的方法。
- ▣ 经验告诉我们，复杂的软件产品有许多错误，但用户一般只能找出这些错误中很少的一部分。
- ▣ 为给用户带来最大利益，要着重对那些用户可能发现的错误进行测试和修改工作。

4 测试类型

- ▣完整性测试：侧重于评估测试对象的强壮性（防止失败的能力），语言、语法的技术兼容性以及资源利用率的测试。该测试针对不同的测试对象实施和执行，包括单元和已集成单元。
- ▣结构测试：侧重于评估测试目标是否符合其设计和构造的测试。通常对基于 Web 的应用程序执行该测试，以确保所有链接都已连接、显示正确的内容以及没有孤立的内容。

测试类型

- 配置测试：侧重于确保测试对象在不同的硬件和/或软件配置上按预期运行的测试。该测试还可以作为系统性能测试来实施。
- 功能测试：侧重于核实测试对象按计划运行，提供需求的服务、方法或用例的测试。该测试针对不同的测试对象实施和执行，包括单元、已集成单元、应用程序和系统。
- 安装测试：侧重于确保测试对象在不同的硬件和/或软件配置上，以及在不同的条件下（磁盘空间不足或电源中断）按预期安装的测试。该测试针对不同的应用程序和系统实施和执行。
- 安全测试：侧重于确保只有预期的主角才可以访问测试对象、数据（或系统）的测试。该测试针对多种测试对象实施和执行。
- 容量测试：侧重于核实测试对象对于大量数据（输入和输出或驻留在数据库内）的处理能力的测试。容量测试包括多种测试策略，如创建返回整个数据库内容的查询；或者对查询设置很多限制，以至不返回数据；或者返回每个字段中最大数据量的数据条目。

测试类型

- ▣ 基准测试：一种性能测试，该测试将比较（新的或未知的）测试对象与已知的参照负载和系统的性能。
- ▣ 竞争测试：侧重于核实测试对象对于多个主角对相同资源（数据记录、内存等）的请求的处理是否可以接受的测试。
- ▣ 负载测试：一种性能测试，用于在测试的系统保持不变的情况下，核实和评估系统在不同负载下操作极限的可接受性。评测包括负载和响应时间的特征。如果系统结合了分布式构架或负载平衡方法，将执行特殊的测试以确保分布和负载平衡方法能够正常工作。
- ▣ 性能测试：在该测试中，将监测测试对象的计时配置文件，包括执行流、数据访问、函数和系统调用，以确定并解决性能瓶颈和低效流程。
- ▣ 强度测试：一种性能测试，侧重于确保系统可在遇到异常条件时按预期运行。系统面对的工作强度可以包括过大的工作量、不充足的内存、不可用的服务/硬件或过低的共享资源。

测试类型

- ▣恢复测试：（例如，硬件故障或用户不良数据引起的一些情况）
- ▣兼容性测试： 对软硬件、操作系统、网络的兼容性测试
- ▣比较测试： 与同类产品进行比较，列出优缺点
- ▣算法测试： 确定是否已正确实现算法
- ▣正向测试： 确定正确使用软件是软件某一特性产生的结果是否与需求一致
- ▣逆向测试： 确定软件无效输入或非法操作的处理是否合理
- ▣时序测试： 测试软件执行的时间序列问题
- ▣突变测试：（mutation testing） 判断测试用例是否有效的方法
- ▣等等

5 测试工具

- 测试用例设计工具
- 录制/回放工具
- 测试覆盖监视工具
- 测试结果比较工具
- 内存泄漏检查工具
- 缺陷跟踪工具
- 性能测试工具

5 测试工具.....

- 测试工具在软件测试中的作用

- 速度

- 自动化测试远高于手工执行测试

- 效率和成本

- 推进软件开发进度、降低开发、测试和维护成本

- 准确度和精度

- 工具能快速且无差错地执行测试用例或测试脚本

5 测试工具.....

- 根据**51test**的调查，当前测试从业人员中最广泛的测试工具为：
 - 功能测试工具：QTP
 - 性能测试工具：LoadRunner
 - 测试管理工具：Quality Center/Test Director
 - 单元测试工具：JUnit
- <http://www.51testing.com/html/index.html>

- QTP: Mercury (美科利) 的自动化功能测试软件QuickTest Professional , 可以覆盖绝大多数的软件开发技术, 简单高效, 并具备测试用例可重用的特点。它自动捕获、验证和重放用户的交互行为。
- LoadRunner: Mercury Interactive公司开发, 是一种预测系统行为和性能的工业标准级负载测试工具。通过以模拟上千万用户实施并发负载及实时性能监测的方式来确认和查找问题, LoadRunner能够对整个企业架构进行测试。
- Test Director: Mercury Interactive公司开发, 是基于WEB的测试管理工具, 能够让你系统地控制整个测试过程, 并创建整个测试工作流的框架和基础, 使整个测试管理过程变得更为简单和有组织。
- JUnit: 是一个Java语言的单元测试框架。它由Kent Beck和Erich Gamma建立, 逐渐成为源于Kent Beck的sUnit的xUnit家族中最为成功的一个。